

```
import numpy as np
import pandas as pd
```

```
df=pd.read_csv("titanic.csv")
```

```
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

```
print(df.tail())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
886	0	2	male	27.0	0	0	13.00	S	Second	
887	1	1	female	19.0	0	0	30.00	S	First	
888	0	3	female	NaN	1	2	23.45	S	Third	
889	1	1	male	26.0	0	0	30.00	C	First	
890	0	3	male	32.0	0	0	7.75	Q	Third	

	who	adult_male	deck	embark_town	alive	alone
886	man	True	NaN	Southampton	no	True
887	woman	False	B	Southampton	yes	True
888	woman	False	NaN	Southampton	no	False
889	man	True	C	Cherbourg	yes	True
890	man	True	NaN	Queenstown	no	True

```
print(df.shape)
```

```
(891, 15)
```

```
print(df.columns)
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
      'alive', 'alone'],  
      dtype='object')
```

```
print(df.isnull().sum())
```

```
survived      0  
pclass        0  
sex           0  
age          177  
sibsp         0  
parch         0  
fare          0  
embarked      2  
class         0  
who           0  
adult_male    0  
deck         688  
embark_town   2  
alive         0  
alone         0  
dtype: int64
```

```
df['age'].fillna(df['age'].mean(),inplace=True)
```

/tmp/ipykernel_4212/3282004082.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which it is performed is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df['age'].fillna(df['age'].mean(),inplace=True)
```

```
print(df.isnull().sum())
```

```
survived      0
pclass        0
sex            0
age           0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
df.dropna(inplace=True)
```

```
print(df[df['age']>30])
```

```
   survived  pclass  sex  age  sibsp  parch  fare  embarked  class \
1          1       1  female  38.0    1     0  71.2833         C  First
3          1       1  female  35.0    1     0  53.1000         S  First
6          0       1   male  54.0    0     0  51.8625         S  First
11         1       1  female  58.0    0     0  26.5500         S  First
21         1       2   male  34.0    0     0  13.0000         S  Second
..      ...     ...   ...   ...    ...   ...   ...     ...   ...
862        1       1  female  48.0    0     0  25.9292         S  First
867        0       1   male  31.0    0     0  50.4958         S  First
871        1       1  female  47.0    1     1  52.5542         S  First
872        0       1   male  33.0    0     0   5.0000         S  First
879        1       1  female  56.0    0     1  83.1583         C  First

   who  adult_male  deck  embark_town  alive  alone
1  woman         False   C   Cherbourg   yes  False
3  woman         False   C  Southampton   yes  False
6   man          True   E  Southampton   no   True
11 woman         False   C  Southampton   yes  True
21  man          True   D  Southampton   yes  True
..  ...     ...   ...   ...    ...   ...   ...
862 woman         False   D  Southampton   yes  True
867  man          True   A  Southampton   no   True
871 woman         False   D  Southampton   yes  False
872  man          True   B  Southampton   no   True
879 woman         False   C   Cherbourg   yes  False
```

```
[113 rows x 15 columns]
```

```
print(df[df['sex']=="female"])
```

```
   survived  pclass  sex  age  sibsp  parch  fare  embarked  \
1          1       1  female  38.000000    1     0  71.2833         C
3          1       1  female  35.000000    1     0  53.1000         S
10         1       3  female  4.000000    1     1  16.7000         S
11         1       1  female  58.000000    0     0  26.5500         S
31         1       1  female  29.699118    1     0  146.5208         C
..      ...     ...   ...   ...    ...   ...   ...     ...
853        1       1  female  16.000000    0     1  39.4000         S
862        1       1  female  48.000000    0     0  25.9292         S
871        1       1  female  47.000000    1     1  52.5542         S
879        1       1  female  56.000000    0     1  83.1583         C
887        1       1  female  19.000000    0     0  30.0000         S

   class  who  adult_male  deck  embark_town  alive  alone
1   First  woman         False   C   Cherbourg   yes  False
3   First  woman         False   C  Southampton   yes  False
10  Third  child         False   G  Southampton   yes  False
11  First  woman         False   C  Southampton   yes  True
31  First  woman         False   B   Cherbourg   yes  False
..  ...     ...   ...   ...    ...   ...   ...
853  First  woman         False   D  Southampton   yes  False
862  First  woman         False   D  Southampton   yes  True
871  First  woman         False   D  Southampton   yes  False
879  First  woman         False   C   Cherbourg   yes  False
887  First  woman         False   B  Southampton   yes  True
```

```
[95 rows x 15 columns]
```

```
print(df.groupby('sex')['survived'].mean())
```

```
sex
female    0.936842
male      0.424528
Name: survived, dtype: float64
```

```
df_sorted=df.sort_values(by="age",ascending=True)
print(df_sorted.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
305	1	1	male	0.92	1	2	151.5500	S	First	
183	1	2	male	1.00	2	1	39.0000	S	Second	
205	0	3	female	2.00	0	1	10.4625	S	Third	
297	0	1	female	2.00	1	2	151.5500	S	First	
340	1	2	male	2.00	1	1	26.0000	S	Second	

	who	adult_male	deck	embark_town	alive	alone
305	child	False	C	Southampton	yes	False
183	child	False	F	Southampton	yes	False
205	child	False	G	Southampton	no	False
297	child	False	C	Southampton	no	False
340	child	False	F	Southampton	yes	False

```
df['Family_size']=df["sibsp"]+df["parch"]
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
1	1	1	female	38.0	1	0	71.2833	C	First	
3	1	1	female	35.0	1	0	53.1000	S	First	
6	0	1	male	54.0	0	0	51.8625	S	First	
10	1	3	female	4.0	1	1	16.7000	S	Third	
11	1	1	female	58.0	0	0	26.5500	S	First	

	who	adult_male	deck	embark_town	alive	alone	Family_size
1	woman	False	C	Cherbourg	yes	False	1
3	woman	False	C	Southampton	yes	False	1
6	man	True	E	Southampton	no	True	0
10	child	False	G	Southampton	yes	False	2
11	woman	False	C	Southampton	yes	True	0

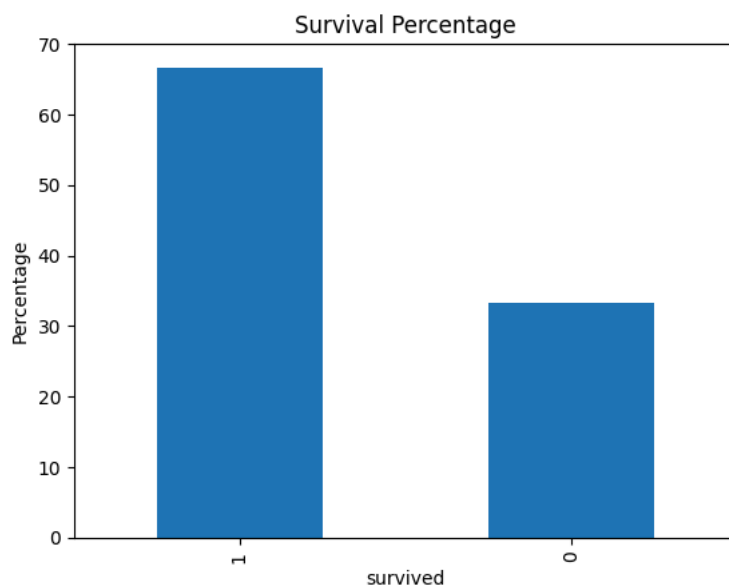
```
print(df['fare'].max())
```

512.3292

```
df.to_csv("cleaned dataset.csv")
```

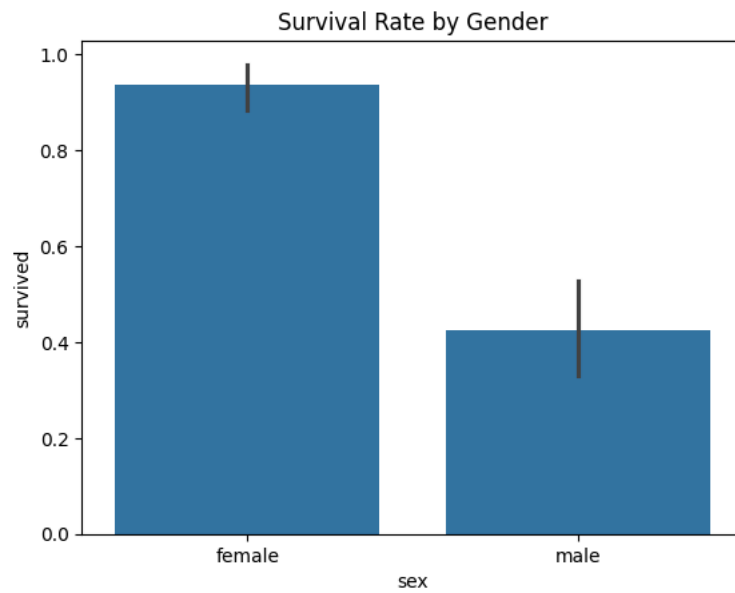
```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
#Survival Rate
survival_rate=df["survived"].value_counts(normalize=True)*100
survival_rate.plot(kind="bar")
plt.title("Survival Percentage")
plt.ylabel("Percentage")
plt.show()
```



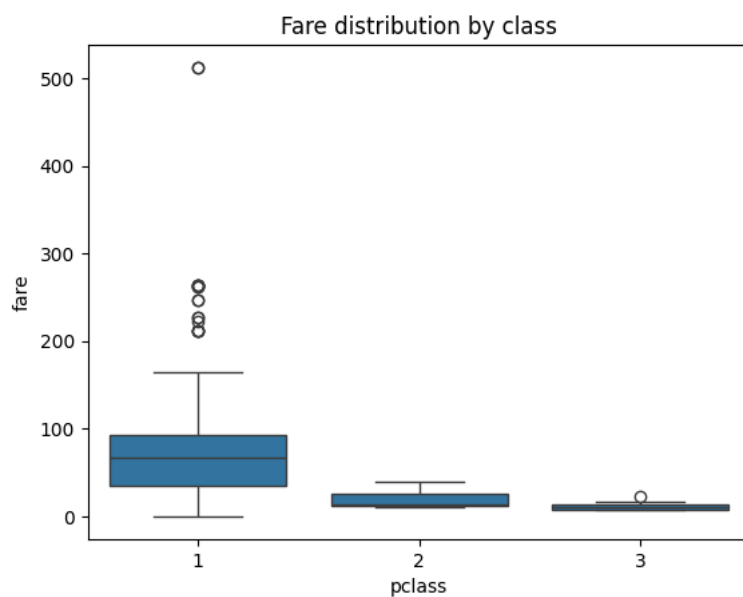
```
# Survival Percentage:65% of people survived
```

```
sns.barplot(x="sex",y="survived",data=df)
plt.title("Survival Rate by Gender")
plt.show()
```



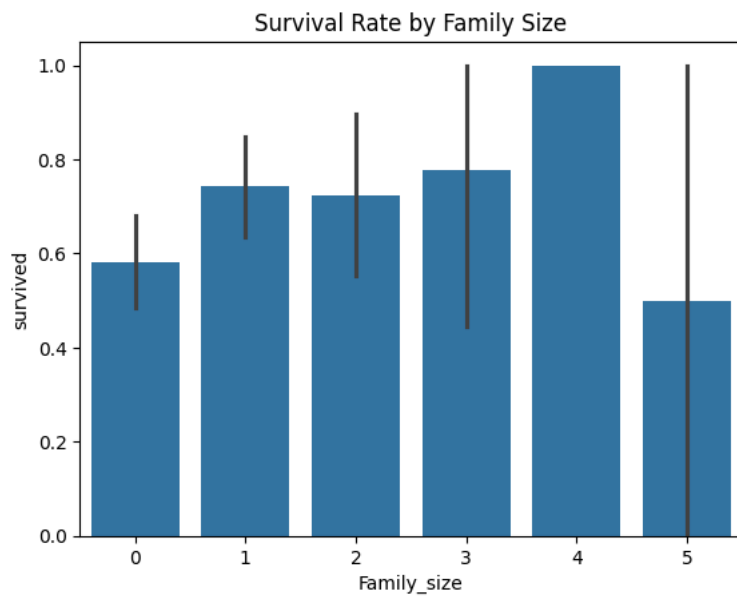
Survival Rate by gender: More females survived then male

```
sns.boxplot(x="pclass",y="fare",data=df)
plt.title("Fare distribution by class")
plt.show()
```

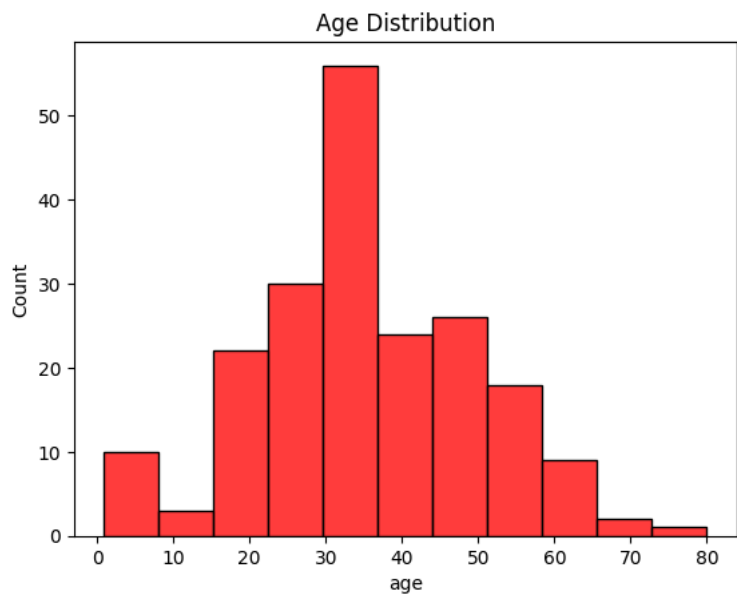


Class 1 people paid more fare

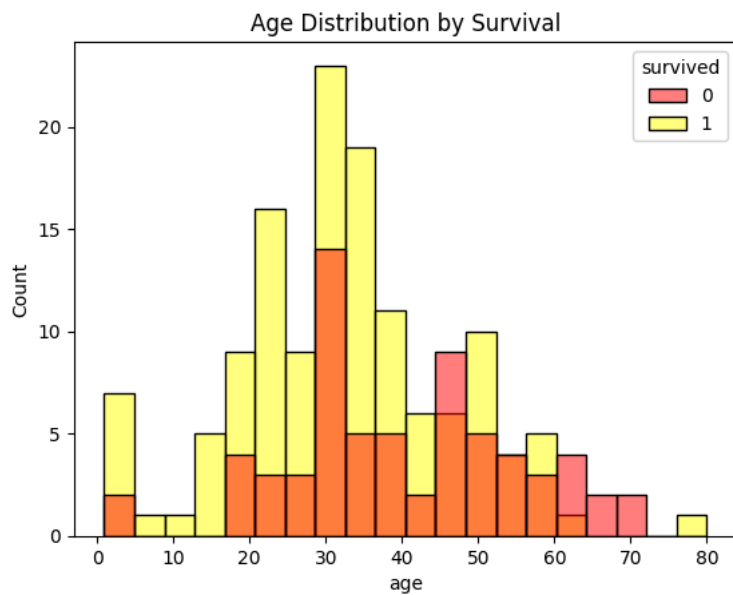
```
df["Family_size"]=df["sibsp"]+df["parch"]
sns.barplot(x="Family_size",y="survived",data=df)
plt.title("Survival Rate by Family Size")
plt.show()
```



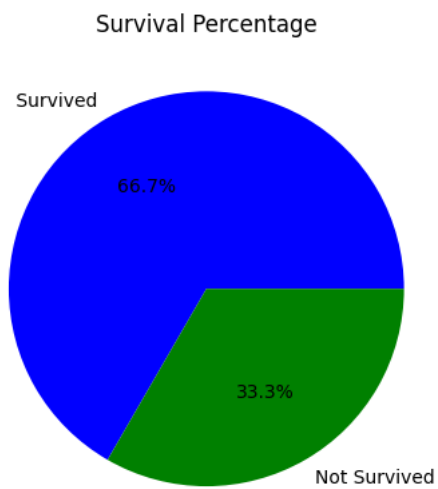
```
sns.histplot(df["age"],color="red")  
plt.title("Age Distribution")  
plt.show()
```



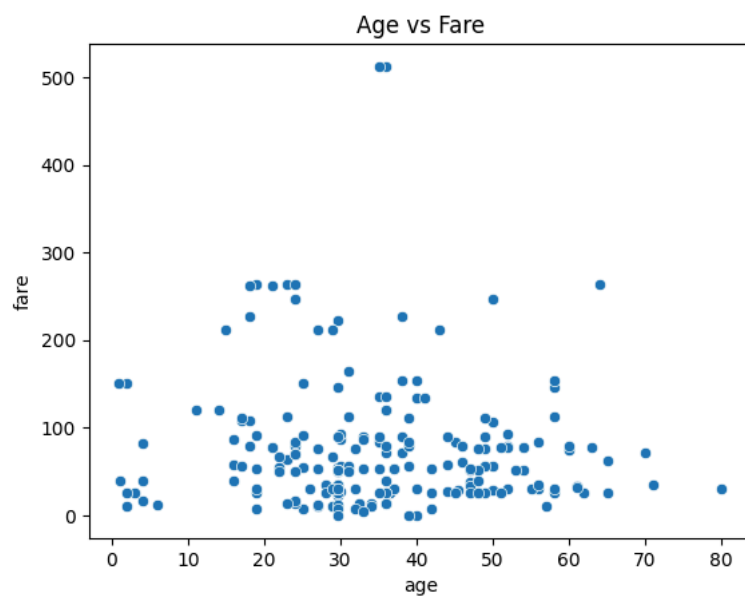
```
sns.histplot(data=df, x="age",hue="survived",palette=["red","yellow"],bins=20)  
plt.title("Age Distribution by Survival")  
plt.show()
```



```
survival=df["survived"].value_counts()
plt.pie(survival,labels=[" Survived","Not Survived"],autopct="%1.1f%%",colors=["blue","green"])
plt.title("Survival Percentage")
plt.show()
```



```
sns.scatterplot(x="age",y="fare",data=df)
plt.title("Age vs Fare")
plt.show()
```



```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

import seaborn as sns
import matplotlib.pyplot as plt

```

```
df = pd.read_csv("titanic.csv")
```

```

df["age"].fillna(df["age"].mean(), inplace=True)
df["embarked"].fillna("S", inplace=True)
df["deck"].fillna("Unknown", inplace=True)

```

/tmp/ipykernel_4212/3924777726.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which the operation is performed is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df["age"].fillna(df["age"].mean(), inplace=True)
```

/tmp/ipykernel_4212/3924777726.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which the operation is performed is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df["embarked"].fillna("S", inplace=True)
```

/tmp/ipykernel_4212/3924777726.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which the operation is performed is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'

```
df["deck"].fillna("Unknown", inplace=True)
```

```
df.drop(["class", "who", "adult_male", "deck",
        "embark_town", "alive"], axis=1, inplace=True)
```

```

df["sex"] = df["sex"].map({"male": 0, "female": 1})
df["embarked"] = df["embarked"].map({"S": 0, "C": 1, "Q": 2})
df["alone"] = df["alone"].map({True: 1, False: 0})

```

```
df["familysize"] = df["sibsp"] + df["parch"]
```

```

x = df.drop("survived", axis=1)
y = df["survived"]

```

```

X_train, X_test, Y_train, Y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

```

```

model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)

```

▼ **LogisticRegression** ⓘ ?
LogisticRegression(max_iter=1000)

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(Y_test, y_pred))
```

```
Accuracy: 0.7988826815642458
```

```

cm = confusion_matrix(Y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

